Universidad de las Américas Puebla

Escuela de ingeniería

Departamento de computación, electrónica y mecatrónica

UDLAP®

Lab report #4

Course: Digital design LRT2022-7

Equipo: 2

183339 Juan Pablo Lopez Moreno LMT
185406 Amy Marianee Ramírez Sánchez LMT

October 20th, 25, San Andrés Cholula, Puebla

1 Abstract

This report will present the results obtained from the forth laboratory practice of the Digital Design course. The main objective of this practice was to begin the development of FPGAs, continuing the use of VHDL as a programming language. The practice focused on the implementation and simulation of combinational logic circuits using both schematic diagrams and Boolean functions.

2 Introduction

Wih the evolution of technologies, digital systems have increasingly incorporated more complex logic behind their functionality, for these reason, technologies have to evolve and improve to suit these needs. For this reason, FPGAs (Field Programmable Gate Arrays) have become a popular choice for implementing digital logic circuits due to their flexibility and reconfigurability. FPGAs allow designers to program custom logic functions directly onto the hardware, enabling rapid prototyping and development of complex digital systems. In order to understand the functioning of the Gate Arrays used, a couple of combinational logic gates will be explored in the lab practice, these gates are as follows:

- Half-substractor, performs the binary subtraction of two single-bit binary numbers. It has two inputs, A and B, and two outputs, Difference and Borrow.
- Multiplexer, combinational circuit that has many data inputs and a single output, depending on control or select inputs.
- Demultiplexer, data distributor combinational circuit. It works in a reverse way of the Multiplexer.
- Magnitud Comparator, combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than, or greater than the other binary number.

[1] It's critical to comprehend not only how these combinational circuits operate but also the various ways in which they can be connected within a program or FPGA. Additionally, we can investigate the plethora of possibilities in the field of digital logic by experimenting with various configurations.

3 Objectives

The objectives of this lab are:

- Program and simulate basic combinational logic circuits VHDL.
- Implement basic combination circuits on the Basys 3 board.

For the purpose of this lab, we must understand the following concepts:

- Combinational Logic Circuits

- VHDL Programming
- FPGA Implementation

Of which, we must begin explaining combiantional logic circuits, which are very well-known components in digital electronics, which can provide an output instantly based on the current input. Unlike sequential circuits, a combinational circuit listens for input signals and generates output regardless of the past input or state, as it has no feedback or memory component.[1] VHDL, which has been used in previous labs, is a hardware description language that allows designers to describe the behavior and structure of digital systems. VHDL is widely used for designing FPGAs and ASICs (Application-Specific Integrated Circuits) due to its ability to model complex digital systems at various levels of abstraction.[2] Finally,field programmable gate array (FPGA) is a versatile type of integrated circuit, which, unlike traditional logic devices such as application-specific integrated circuits (ASICs), is designed to be programmable (and often reprogrammable) to suit different purposes, notably high-performance computing (HPC) and prototyping.[3] FPGA implementation involves programming the FPGA device with the designed VHDL code to realize the desired combinational logic circuit. This process includes synthesis, mapping, placement, and routing of the design onto the FPGA fabric.

4 Methodology

The methodology applied in this laboratory practice was structured into three main phases: logical analysis, simulation in EDA Playground, and physical implementation on the Basys 3 board.

4.1 Logical Analysis Phase

During this first stage, the following activities were carried out:

1. The truth tables corresponding to the half-subtractor, multiplexer, magnitude comparator, and demultiplexer circuits, all of one bit, were obtained.
2. Based on the truth tables, the corresponding Karnaugh maps were developed in order to simplify the Boolean expressions and obtain the minimal logical function for each circuit.
3. Finally, the obtained results were documented for use in the subsequent simulation and programming phases.

4.2 EDA Playground Simulation Phase

In the EDA Playground digital simulation environment, the following procedures were performed:

1. The half-subtractor, multiplexer, magnitude comparator, and demultiplexer circuits were programmed using the VHDL language.
2. Testbenches were developed to verify the behavior of each circuit under all possible input combinations.

3. The results obtained from the simulations were analyzed and compared with the previously developed truth tables and simplified functions, ensuring the correct logical implementation of each design.

4.3 Basys 3 Board Implementation Phase

Finally, the physical implementation of the circuits was carried out on the Basys 3 board, following the steps described below:

1. The code for each circuit was synthesized, and the corresponding bitstream files were generated.
2. Pin constraints (`.xdc` file) were assigned to map the logical signals to the physical components of the board (switches, LEDs, and displays).
3. The designs were loaded onto the Basys 3 board, verifying their correct operation through practical input and output tests.

5 Results

In this lab, the physical circuits were not built, but based on certain truth tables, EDAWaves were generated in order to accomplish every single combinational circuit given.:

5.1 Half-substractor

For the half-substractor, the following truth table was used to generate the EDA Code:

Input		**Output**	
A	**B**	**AC**	**R**
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 1: Truth Table for the Half-substractor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 1: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_half is
port(
```

```
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    AC : out std_logic;
    R : out std_logic);     -- OR gate output

end ent_half;

-- Architecture definition

architecture arq_half of ent_half is

 begin

    AC <= (not(A)and(B));
    R <= ((not(A)and(B))or((A)and(not(B))));

end arq_half;
```

With it's appropriate testbench:

Código 2: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_half is
port(
        A : in std_logic;       -- OR gate input
    B : in std_logic;
    AC : out std_logic;
    R : out std_logic);
end component;

signal a_in, b_in, ac_out, r_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_half port map(a_in, b_in, ac_out, r_out);
```

```
  process
  begin

    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;

        a_in <= '0';
    b_in <= '1';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '0';
    wait for 1 ns;
    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

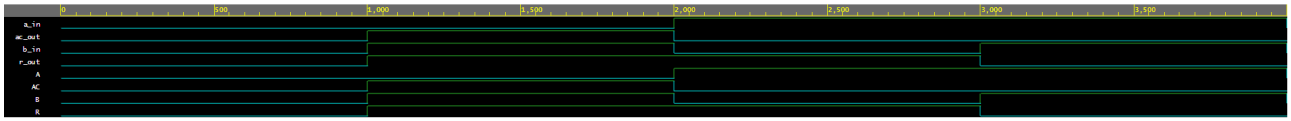

Figure 1: Wave generated for the Half-substractor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

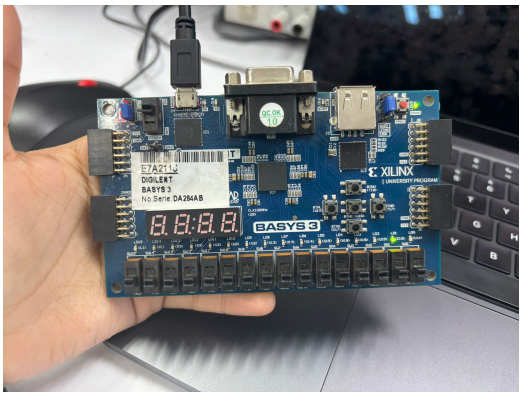

Figure 2: Evidence of the functioning of the Half-substractor in the Basys 3 board.

5.2 Multiplexor

For the multiplexor, the following truth table was used to generate the EDA Code:

Input			**Output**
Se	I_1	I_0	**out**
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 2: Truth Table for the 2-to-1 Multiplexor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 3: Code 1: For exercise 2

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_mult is
port(
    SE : in std_logic;        -- OR gate input
    I1 : in std_logic;
    I0 : in std_logic;
    Y : out std_logic);      -- OR gate output

end ent_mult;

-- Architecture definition

architecture arq_mult of ent_mult is

 begin

    Y <= ((not(SE)and(I0)))or((SE)and(I1));

end arq_mult;
```

With it's appropriate testbench:

Código 4: Code 2: Testbench for exercise 2

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_mult is
port(
    SE : in std_logic;       -- OR gate input
    I1 : in std_logic;
    I0 : in std_logic;
    Y : out std_logic);
end component;

signal se_in, i1_in, i0_in, y_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_mult port map(se_in, i1_in, i0_in, y_out);

  process
  begin

    se_in <= '0';
    i1_in <= '0';
    i0_in <= '0';
    wait for 1 ns;

        se_in <= '0';
    i1_in <= '0';
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '0';
    i1_in <= '1';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '0';
    i1_in <= '1';
```

```
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '1';
    i1_in <= '0';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '1';
    i1_in <= '0';
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '1';
    i1_in <= '1';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '1';
    i1_in <= '1';
    i0_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

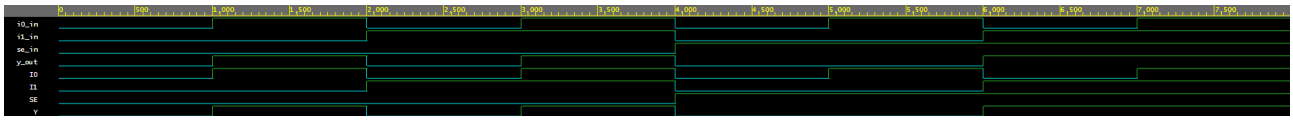

Figure 3: Wave generated for the Multiplexor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

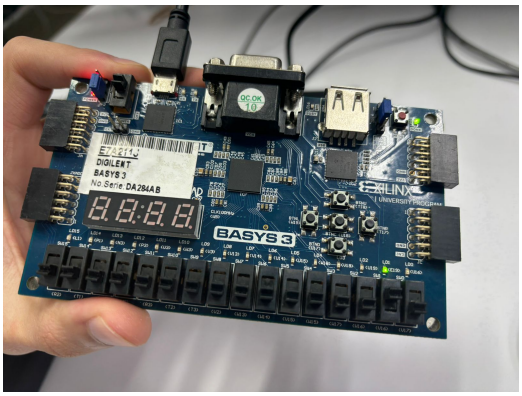

Figure 4: Evidence of the functioning of the Multiplexor in the Basys 3 board.

5.3 Demultiplexor

For the demultiplexor, the following truth table was used to generate the EDA Code:

Input		**Output**	
Se	I_0	O_1	O_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

Table 3: Truth Table for the 1-to-2 Demultiplexor

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 5: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_demult is
port(
    SE : in std_logic;          -- OR gate input
    I0 : in std_logic;
    O1 : out std_logic;
    O0 : out std_logic);        -- OR gate output

end ent_demult;

-- Architecture definition

architecture arq_demult of ent_demult is
```

```
 begin

    O1 <= (((SE)and(I0)));
    O0 <= ((not(SE)and(I0)));


end arq_demult;
```

With it's appropriate testbench:

Código 6: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_demult is
port(
    SE : in std_logic;       -- OR gate input
    I0 : in std_logic;
    O1 : out std_logic;
    O0 : out std_logic);     -- OR gate output

end component;

signal se_in, i0_in, O1_out, O0_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_demult port map(se_in, i0_in, O1_out, O0_out);

  process
  begin

    se_in <= '0';
    i0_in <= '0';
    wait for 1 ns;

        se_in <= '0';
```

```
    i0_in <= '1';
    wait for 1 ns;

    se_in <= '1';
    i0_in <= '0';
    wait for 1 ns;

    se_in <= '1';
    i0_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

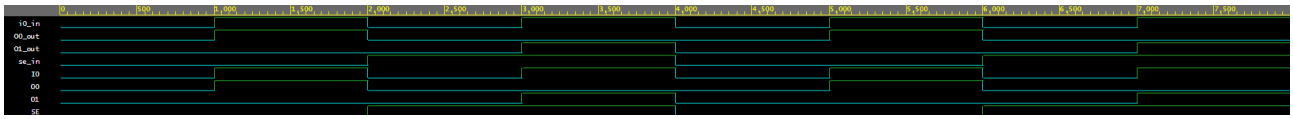

Figure 5: Wave generated for the Demultiplexor.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

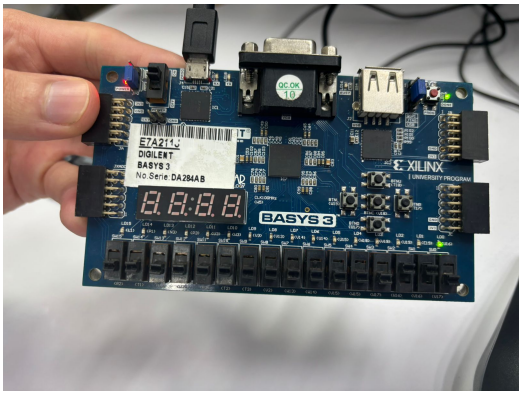

Figure 6: Evidence of the functioning of the Demultiplexor in the Basys 3 board.

5.4 Magnitud Comparator

For the comparator, the following truth table was used to generate the EDA Code:

Input		Output		
A	**B**	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Table 4: Truth Table for a 1-bit Magnitude Comparator

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 7: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration

entity ent_comp is
port(
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    IG : out std_logic;
    MI : out std_logic;
    MA : out std_logic);     -- OR gate output

end ent_comp;

-- Architecture definition

architecture arq_comp of ent_comp is

 begin

    MA <= (A)and(not(B));
    IG <= ((not(A))and(not(B)))or((A)and(B));
    MI <= (B)and(not(A));

end arq_comp;
```

With it's appropriate testbench:

Código 8: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
```

```
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_comp is
port(
    A : in std_logic;       -- OR gate input
    B : in std_logic;
    IG : out std_logic;
    MI : out std_logic;
    MA : out std_logic);     -- OR gate output
end component;

signal a_in, b_in, ig_out, mi_out, ma_out: std_logic;

begin

  -- Connect UUT
  UUT: ent_comp port map(a_in, b_in, ig_out, mi_out, ma_out);

  process
  begin

    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;

        a_in <= '0';
    b_in <= '1';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '0';
    wait for 1 ns;

    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;

  end process;
end tb;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

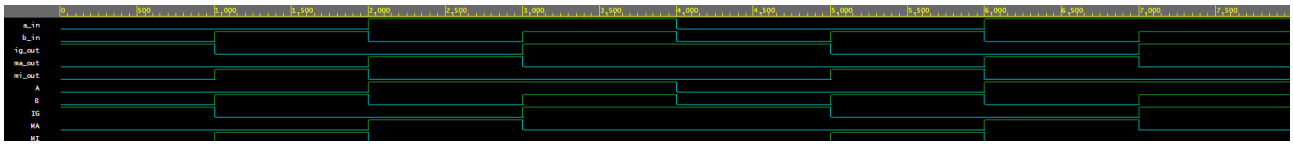

Figure 7: Wave generated for the Magnitud Comparator.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

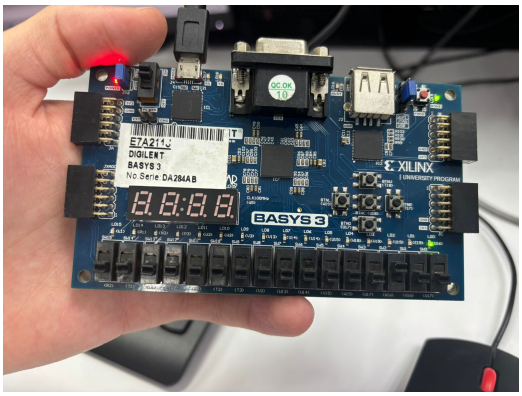

Figure 8: Evidence of the functioning of the Magnitud Comparator in the Basys 3 board.

6 Analysis of Results

In this laboratory practice, four fundamental digital circuits were analyzed—half-subtractor, 2:1 multiplexer, 1→2 demultiplexer, and 1-bit magnitude comparator—following three stages: logical analysis through truth tables, simulation in EDA Playground using VHDL, and practical verification on the Basys 3 board. The observed behavior in each phase is described in detail below.

6.1 Half-Subtractor

6.1.1 Truth Table Analysis

The half-subtractor possesses inputs A (minuend) and B (subtrahend) and generates two outputs: R, representing the logical difference $(A-B)$, and AC, which indicates whether a borrow was required. When both inputs are zero, no difference or borrow occurs, and both outputs remain inactive. If $A = 0$ and $B = 1$, subtracting a larger number from a smaller one yields a difference (R) equal to one and activates the borrow $(AC = 1)$, reflecting the necessity of borrowing from the higher-order bit. When $A = 1$ and $B = 0$, no borrow is required, and the difference equals one, activating only R. Finally, when $A = 1$ and $B = 1$, the inputs are equal, the subtraction results in zero, and no output is activated. The observed behavior confirms that R functions as an XOR operation and AC as an AND operation with inversion of the minuend.

6.1.2 EDA Playground Analysis

During simulation, the generated waveforms matched the expected values precisely. The R output activated only when the inputs differed, while AC activated exclusively when A was less than B. State transitions occurred without delays or interference, validating the correct implementation of the logical equations in the VHDL code.

6.1.3 Basys 3 Implementation

On the Basys 3 board, switches were employed to represent inputs A and B, while LEDs displayed outputs AC and R. Physical testing demonstrated that the LED corresponding to R illuminated when the inputs differed, and AC lit solely when subtracting 1 from 0. The observed behavior fully coincided with simulation and theoretical expectations, demonstrating correct and stable implementation.

6.2 2:1 Multiplexer

6.2.1 Truth Table Analysis

The 2:1 multiplexer features three inputs: the selection signal Se and data inputs $I1$ and $I0$. Its output, *out*, reflects the value of one of the two inputs depending on the state of Se. When $Se = 0$, the output adopts the value of $I0$, disregarding $I1$. Conversely, when $Se = 1$, the output assumes the value of $I1$. This behavior confirms that the output is solely dependent on the selector.

6.2.2 EDA Playground Analysis

During simulation, the waveforms exhibited precise switching between data inputs as Se varied. When the selector was low, the output followed $I0$; when high, it followed $I1$. No overshoots or indeterminate signals were observed, validating the correct VHDL implementation.

6.2.3 Basys 3 Implementation

In the physical test, the board switches controlled Se, $I1$, and $I0$, and the LED displayed the output. The observed results aligned with theoretical expectations: with the selector at 0, the output reproduced $I0$; with the selector at 1, it reproduced $I1$. The response was immediate, without switching errors, confirming the accuracy of the logical design and proper pin assignment.

6.3 1→2 Demultiplexer

6.3.1 Truth Table Analysis

The demultiplexer receives the selection signal Se and data input $I0$, generating two outputs: $O1$ and $O0$. When $Se = 0$, the input data is directed exclusively to $O0$, while $O1$ remains low. When $Se = 1$, the signal is transferred to $O1$, and $O0$ is deactivated. This confirms that the circuit directs the input to a single active output based on the selector.

6.3.2 EDA Playground Analysis

Simulation verified that both outputs were never active simultaneously. When the selector changed, the active output correctly switched between $O0$ and $O1$ according to input $I0$. Waveforms demonstrated clean transitions, free of residual signals or timing errors.

6.3.3 Basys 3 Implementation

For the physical implementation, switches were assigned to Se and $I0$, and LEDs indicated outputs $O1$ and $O0$. Testing confirmed that when the selector was low, only $O0$ responded to the input; when high, the response transferred to $O1$. These results corroborated the correct distribution of the signal and the fidelity of the logical design relative to the simulation.

6.4 1-bit Magnitude Comparator

6.4.1 Truth Table Analysis

The magnitude comparator receives two input bits, A and B, and produces three outputs: $A > B$, $A = B$, and $A < B$. When both inputs are zero, they are equal, activating the $A = B$ output. If $A = 0$ and $B = 1$, A is smaller, activating $A < B$. If $A = 1$ and $B = 0$, A is greater, activating $A > B$. Finally, when both inputs are 1, $A = B$ is activated again. In all cases, only one output remains active, confirming the exclusive nature of the comparator's design.

6.4.2 EDA Playground Analysis

Simulation waveforms exhibited clear and stable behavior. Each output responded exclusively to the corresponding relationship between inputs, without overlapping signals. The circuit reacted instantly to changes in A and B, demonstrating precise logic without coding errors.

6.4.3 Basys 3 Implementation

In the physical test, switches represented inputs A and B, and three LEDs displayed outputs $A > B$, $A = B$, and $A < B$. Observations indicated that each LED activated solely under its corresponding condition: "greater" when A exceeded B, "equal" when inputs matched, and "less" when B exceeded A. The complete agreement between simulation and practice confirmed the correct synthesis and mapping of the comparator.

7 Conclusion

This laboratory practice allowed for the analysis, simulation, and implementation of four fundamental digital circuits: half-subtractor, 2:1 multiplexer, 1→2 demultiplexer, and 1-bit magnitude comparator. By comparing theoretical results, simulations in EDA Playground, and physical verification on the Basys 3 board, it was confirmed that all circuits functioned according to the intended designs. The analysis of truth tables enabled a clear understanding of each circuit's logical behavior, while digital simulation validated the correct VHDL implementation, showing clean transitions and the absence of timing errors or interference. The Basys 3 implementation demonstrated the fidelity of the design by physically reproducing the expected

operations using switches and LEDs, confirming the circuits' stability and accuracy. In conclusion, this practice reinforced the understanding of fundamental digital logic principles, the importance of verification through simulation, and the relevance of physical implementation to validate theoretical designs. Additionally, it strengthened skills in using digital design tools, such as EDA Playground and FPGA boards, which are essential for developing reliable and efficient digital systems.

References

[1] GeeksforGeeks. (2025) What is combinational circuit ? [En línea]. Disponible: https://www.geeksforgeeks.org/digital-logic/what-is-combinational-circuit

[2] Intel Corporation, "Vhdl definition," https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm, 2017, accessed: 2024-10-27.

[3] J. Schneider and I. Smalley. (2025) Field programmable gate array. [En línea]. Disponible: https://www.ibm.com/think/topics/field-programmable-gate-arrays